

Bronze Layer Documentation

DATA ENGINEERING PROJECT MEDALLION ARCHITECTURE

This document provides a comprehensive technical reference for the Bronze Layer implementation within a university Data Engineering project. It covers the architectural purpose, data source specifications, Snowflake configuration, data loading workflow, and validation results. The Bronze Layer serves as the foundational ingestion point in the Medallion Architecture, capturing raw data exactly as received from the source system without any transformations applied.

Overview & Data Source

The Medallion Architecture is a data design pattern used to organize data in a lakehouse environment into

three distinct quality layers: Bronze, Silver, and Gold. Each layer serves a specific purpose in the data refinement pipeline, progressively improving data quality, structure, and business readiness. The Bronze Layer is the first and most foundational tier in this architecture. It acts as the raw ingestion zone where data is stored exactly as it arrives from the source system — with no transformations, filtering, or cleansing applied. This approach preserves the original fidelity of the data and provides a reliable audit trail for downstream processing and debugging.

The primary purpose of the Bronze Layer is to create a durable, immutable copy of source data. By storing data in its original form, data engineers can replay transformation pipelines, investigate data quality issues, and maintain compliance with data governance requirements. In this project, the Bronze Layer is implemented within Snowflake, a cloud-based data warehousing platform, using a structured set of database objects including a database, warehouse, schema, stage, file format, and table.

The dataset used in this project is a real estate listings file containing property information such as location, price, area, and other attributes relevant to analytical processing. The data is provided in CSV format and serves as the input for the Bronze Layer ingestion pipeline. Once loaded into Snowflake, this raw data becomes the foundation for subsequent Silver Layer transformations (cleansing, deduplication, standardization) and Gold Layer aggregations (business-level metrics and summaries).

The choice of Snowflake as the platform for this implementation is deliberate. Snowflake's separation of storage and compute, its native support for semi-structured data, and its powerful COPY INTO command make it an ideal choice for building scalable medallion architectures. The platform's ability to handle large volumes of data with minimal configuration overhead allows students to focus on architectural patterns rather than infrastructure complexity. Additionally, Snowflake's time-travel and zero-copy cloning features provide additional safeguards for data integrity at the Bronze Layer.

The Bronze Layer table in this implementation is named RAW_REAL_ESTATE and resides within the BRONZE schema of the REAL_ESTATE_DB database. The table schema is designed to accommodate all columns present in the source CSV file, with data types mapped appropriately to Snowflake's type system. No primary keys, foreign keys, or constraints are applied at this stage, as the goal is to preserve the raw structure of the incoming data exactly as it was received from the source.

Data Source Specifications

The following table summarizes the key attributes of the source dataset used for Bronze Layer ingestion. The dataset is a real estate listings file in CSV format, containing property records intended for downstream analytical processing. Each row represents a single property listing with attributes such as location, price, area, and other relevant fields. The dataset was validated for completeness prior to ingestion to ensure all 2,060 records could be loaded without errors.

File Name	real-estate-raw.csv
Format	CSV (Comma-Separated Values)
Rows	2,060
Loaded	Real estate listings for analytical processing
Purpose	

The CSV format was chosen for its simplicity and wide compatibility with data loading tools. Snowflake's native COPY INTO command supports CSV ingestion with configurable file format options, including field delimiters, escape characters, and null value handling. The CSV_FORMAT file format object defined in Snowflake specifies these options to ensure consistent parsing of the source file during the load operation.

Snowflake Configuration

The Bronze Layer implementation relies on a structured set of Snowflake objects, each serving a distinct role in the data ingestion pipeline. The configuration follows Snowflake best practices for organizing cloud data warehouse environments, with clear separation of concerns across database, compute, schema, and storage abstraction layers. The hierarchy of these components is important to understand: the database serves as the top-level container, the warehouse provides the compute engine, schemas organize objects logically within the database, stages act as temporary storage areas for files awaiting ingestion, file formats define how Snowflake should interpret incoming files, and tables store the final loaded data.

Each component in this configuration was created using standard Snowflake DDL (Data Definition Language) statements. The database and warehouse were created first to establish the foundational environment. The BRONZE schema was then created within the database to provide a dedicated namespace for all Bronze Layer objects. The stage and file format were defined next to prepare for file ingestion, and finally, the target table was created with a schema matching the source CSV structure. This order of operations ensures that all dependencies are satisfied before each subsequent step is executed.

The REAL_ESTATE_WH warehouse was configured with appropriate sizing for this project's data volume. In Snowflake, warehouses are independent compute clusters that can be scaled up or down based on workload requirements. For a dataset of 2,060 rows, a small or medium-sized warehouse is sufficient, though the specific T-shirt size (e.g., X-Small, Small) can be adjusted based on performance testing. The warehouse is set to auto-suspend after a period of inactivity to optimize cost efficiency, a key consideration in cloud data warehouse management.

The RAW_STAGE internal stage provides a secure, managed location within Snowflake to temporarily hold the CSV file before loading. Internal stages are stored within Snowflake's storage layer and are accessible only to authorized roles, ensuring that sensitive data is not exposed during the ingestion process. The stage acts as a bridge between the external file system (where the CSV is initially uploaded) and the target table, allowing Snowflake to read and parse the file during the COPY INTO operation.

Component	Name	Purpose
Database	REAL_ESTATE_DB	Top-level container for all project objects
Warehouse	REAL_ESTATE_WH	Compute resource for query execution
Schema	BRONZE	Logical namespace for Bronze Layer tables
Stage	RAW_STAGE	Temporary storage for files before loading
File Format	CSV_FORMAT	Defines CSV parsing rules for ingestion
Table	RAW_REAL_ESTATE	Target table storing raw ingested data

The separation of these components follows Snowflake's recommended architectural patterns. By isolating the Bronze Layer in its own schema, the implementation ensures clear boundaries between raw, cleaned, and aggregated data tiers. This separation simplifies access control, as different user roles can be granted permissions to specific schemas based on their responsibilities. For example, data engineers may have write access to the Bronze schema, while analysts may only have read access to the Silver and Gold schemas.

The CSV_FORMAT file format object encapsulates all parsing configuration required for the source file. This includes settings such as the field delimiter (comma), record delimiter (newline), escape character handling, and null value representation. By defining the file format as a named object in Snowflake, it can be reused across multiple COPY INTO operations, promoting consistency and reducing configuration errors. The file format is referenced by name during the data load command, ensuring that Snowflake applies the correct parsing rules to the incoming CSV file.

Data Loading Process

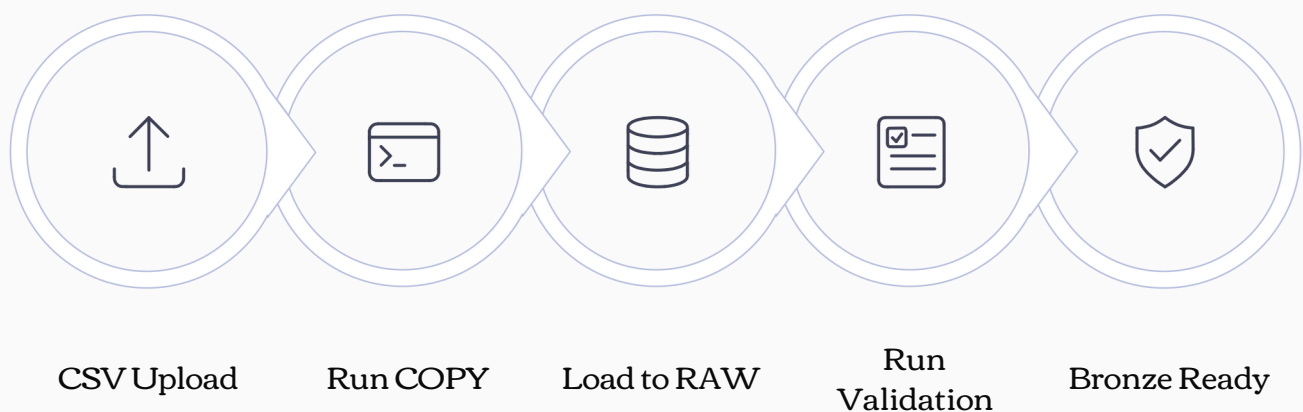
The data loading process for the Bronze Layer follows a structured, nine-step workflow that ensures all Snowflake objects

are properly configured before data ingestion begins. This sequential approach minimizes the risk of dependency errors and provides clear checkpoints for validation at each stage. The workflow is designed to be repeatable, allowing the pipeline to be re-executed in case of source data updates or environment resets.

The first step involves creating the `REAL_ESTATE_DB` database, which serves as the top-level container for all project objects. Without a database, no other Snowflake objects can be created. The second step creates the `REAL_ESTATE_WH` warehouse, which provides the compute resources necessary for executing DDL and DML operations. The warehouse must be active (or set to auto-resume) before any data loading commands can be executed.

Steps three through five establish the logical and physical infrastructure for data ingestion. The `BRONZE` schema is created within the database to provide a dedicated namespace. The `RAW_STAGE` internal stage is then created within the schema to serve as the temporary holding area for the CSV file. Finally, the `CSV_FORMAT` file format object is defined with the appropriate parsing options for the source file. These three objects must exist before the `COPY INTO` command can be executed successfully.

Steps six through nine cover the actual data loading and validation. The CSV file is uploaded to the stage using Snowflake's `PUT` command (for local files) or referenced directly from cloud storage. The `COPY INTO` command then reads the file from the stage, parses it according to the specified file format, and inserts the rows into the `RAW_REAL_ESTATE` table. Finally, validation queries confirm that the expected number of rows were loaded without errors.



The diagram above illustrates the core data flow from the source CSV file through the Snowflake ingestion pipeline to the final validated Bronze Layer table. Each arrow represents a distinct operation that must complete successfully before the next step can begin. The `COPY INTO` command is the central operation in this workflow, as it performs the actual data transfer from the stage to the target table.

The `COPY INTO` command in Snowflake is a bulk data loading utility that efficiently transfers data from a stage into a target table. It supports parallel loading, automatic file parsing, and detailed error reporting. The command syntax for this implementation specifies the target table (`RAW_REAL_ESTATE`), the source stage (`@RAW_STAGE`), and the file format (`CSV_FORMAT`). Additional options such as `ON_ERROR = CONTINUE` or `ON_ERROR = ABORT_STATEMENT` can be configured to control how Snowflake handles malformed rows during the load.

One of the key advantages of the `COPY INTO` command is its ability to handle large files efficiently by splitting them into micro-partitions and loading them in parallel. For this project's dataset of 2,060 rows, the load completes almost instantaneously, but the same command scales to handle millions of rows without modification. The command also generates a load summary report that includes the number of rows loaded, rows skipped, and any errors encountered, which is essential for the validation step.

The validation step is critical to ensure data integrity before proceeding to Silver Layer transformations. Three primary validation queries are executed: `SELECT COUNT(*)` to verify the total row count matches the expected 2,060 rows, `SELECT * LIMIT 20` to visually inspect a sample of the loaded data, and `DESC TABLE RAW_REAL_ESTATE` to confirm the table schema matches the source file structure. These checks provide confidence that the Bronze Layer is correctly populated and ready for downstream processing.

Validation & Conclusion

Following the data loading process, a series of validation checks were performed to confirm successful ingestion of the real estate dataset into the Bronze Layer table. These checks are standard practice in data engineering pipelines to ensure data integrity before proceeding to downstream transformations. The validation process verifies three key aspects: the completeness of the load (all rows were ingested), the correctness of the data (rows match the source), and the structural integrity of the target table (schema matches expectations).

The first validation query, `SELECT COUNT(*)`, returns the total number of rows in the `RAW_REAL_ESTATE` table. This count is compared against the expected row count from the source CSV file to ensure no rows were skipped or duplicated during the load. The second query, `SELECT * LIMIT 20`, retrieves the first 20 rows of the table for visual inspection. This allows the data engineer to verify that column values are correctly parsed, that delimiters were handled properly, and that no unexpected null values or formatting issues are present. The third query, `DESC TABLE RAW_REAL_ESTATE`, displays the table schema including column names, data types, and nullability constraints, confirming that the table structure matches the source file.

2060	2060	0
Rows Parsed		Errors
Total rows identified in the source CSV file during the COPY INTO operation	All rows successfully inserted into the RAW_REAL_ESTATE table without loss	

No parsing errors, rejections, or warnings reported during ingestion

The validation results confirm a 100% successful data ingestion. All 2,060 rows from the source CSV file were parsed and loaded into the `RAW_REAL_ESTATE` table with zero errors. This perfect load rate indicates that the source file was well-formed, the `CSV_FORMAT` file format was correctly configured, and the `COPY INTO` command executed without encountering any malformed rows or data type mismatches. The absence of errors is particularly important in a production context, as even a small number of rejected rows could indicate data quality issues that would propagate through downstream transformations.

The row count validation is especially critical in the Medallion Architecture context. Since the Bronze Layer serves as the authoritative source of raw data, any discrepancy between the source file and the loaded table would undermine the reliability of all downstream layers. The matching counts of 2,060 rows parsed and 2,060 rows loaded provide strong evidence that the ingestion pipeline is functioning correctly and that the Bronze Layer is ready to serve as the foundation for Silver Layer transformations.

Conclusion

The Bronze Layer has been successfully implemented within the Snowflake environment, establishing a robust and reliable foundation for the Medallion Architecture data pipeline. All configuration components — including the database, warehouse, schema, stage, file format, and target table — have been created and validated. The raw real estate dataset comprising 2,060 records has been ingested without errors, preserving the original data fidelity as required by Bronze Layer design principles.

The successful completion of the Bronze Layer marks a critical milestone in the data engineering project. With raw data now securely stored in Snowflake, the pipeline is ready to advance to the Silver Layer, where data cleansing, deduplication, standardization, and type casting transformations will be applied. The immutable nature of the Bronze Layer ensures that the original source data remains available for audit, replay, and debugging purposes throughout the project lifecycle. The architectural patterns established in this layer — including schema separation, named file formats, and structured validation — provide a reusable template for future data ingestion tasks within the project.

The implementation demonstrates key data engineering concepts including cloud data warehouse configuration, staged data ingestion, bulk loading with `COPY INTO`, and systematic validation. These patterns are directly applicable to enterprise-scale data pipelines and provide a solid foundation for understanding more advanced topics such as incremental loading, schema evolution, and data quality monitoring. The Bronze Layer documentation serves as both a technical reference and a learning artifact for students and faculty involved in the project.

Next Steps: Proceed to Silver Layer implementation — apply data cleansing, deduplication, standardization, and type casting transformations to prepare data for analytical consumption.